Project Design Phase-II
Data Flow Diagram & User Stories

Date	27.03.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

1. User Stories

User stories help define the "who, what, and why" of the requirement.

ID	Role	Requirement	Acceptance Criteria
US.01	**IT Agent**	I want the "Resolution Notes" to be mandatory when an Incident is resolved.	Field becomes mandatory only when state changes to "Resolved."
US.02	**Developer**	I want to enforce field requirements via UI Policy rather than Client Script.	Logic runs on the client-side without complex coding; UI Policy Actions are used.
US.03	**Admin**	I want to capture these UI changes in an Update Set for migration.	All UI Policies and Policy Actions are tracked in the current Update Set.

2. Data Flow Diagram (DFD)

The following DFD illustrates how a UI Policy interacts with the user's browser and how those configurations move through the system landscape via Update Sets.

The Flow Explained:

- **Trigger:** The user changes a value on the form (e.g., State moves to "Resolved").
- **Evaluation:** The **UI Policy engine** in the browser checks the condition locally without a server call.
- **Execution:** If the condition is `true`, the **UI Policy Action** is triggered.
- **UI Modification:** The DOM is updated to display the mandatory asterisk (*).
- **Capture:** As the Admin configures this, the **Update Set** records the XML definition of the Policy.
- **Migration:** The Update Set is closed and moved from **Dev → QA → Prod**.

```
[User Opens Form]
      ↓
[Form Loads in UI]
      ↓
[UI Policy Triggered]
      ↓
[Condition Checked]
  ├── If TRUE → [Set Field Mandatory]
  |             ↓
  |       [User Must Fill Field]
  |             ↓
  |       [Form Submitted]
  |
  └── If FALSE → [Field Not Mandatory]
                ↓
          [Form Submitted]

      ↓
[Changes Saved in Instance]
      ↓
[Create/Update Update Set]
      ↓
[Capture UI Policy Changes]
      ↓
[Move Update Set to Target Instance]
      ↓
[Preview & Resolve Conflicts]
      ↓
[Commit Update Set]
      ↓
[Changes Applied in Target Instance]
```

3. Implementation Steps

Part A: Enforcing Mandatory Fields

To make a field mandatory based on a condition:

- **Navigate to:** `System UI > UI Policies`.
- **Condition:** Set your trigger (e.g., `[State] [is] [Resolved]`).
- **Global/View:** Ensure "Global" is checked so it applies to all views.
- **UI Policy Action:** Create a new record in the related list:
  - **Field name:** `u_resolution_notes`
  - **Mandatory:** `True`

Part B: Moving Changes with Update Sets

Since UI Policies are configuration data (stored in the `sys_ui_policy` table), they are automatically captured.

- **Set the Scope:** Ensure you are in the correct Application Scope (e.g., Global or a scoped app).
- **Create Set:** Go to `Local Update Sets > New`. Name it (e.g., `INC_Mandatory_Fields_v1`).
- **Make "Current":** Click **Make This My Current Set**.
- **Configure:** Perform the UI Policy steps mentioned in Part A.
- **Review:** Check the `Customer Updates` related list at the bottom of your Update Set to ensure your Policy and Actions are listed.
- **Complete & Export:** Set the state to **Complete**, then use **Export to XML** or **Retrieve** from the target instance.

4. Best Practices

- **UI Policies > Client Scripts:** Always use a UI Policy for "Mandatory," "Read Only," or "Visible" requirements. It's faster and requires zero code.
- **Reverse if False:** Ensure the "Reverse if false" checkbox is checked so the field becomes non-mandatory if the condition is no longer met.
- **Naming Conventions:** Name your Update Sets clearly (e.g., `STORY101_UIPolicy_Refactor`) to make troubleshooting easier in the production logs.